Approximation Algorithms for Multicommodity Flows on Trees

by

Marcus Lai

Abstract

We study the maximum integer multicommodity flow problem on trees where we are given a tree $T = (V, E)$ with integer capacities and a graph $H = (V, F)$ that encodes the commodities we may select to route. While the unit capacity case can be solved in polynomial time, the problem is APX-hard if we allow capacities in $\{1, 2\}$. As such, we present a 2-approximation for the general (integer) capacity case by Garg et al. One extension is to the maximum multicommodity profit flow problem where commodities have associated profits and the goal is to maximize the all-or-nothing profit (profits are only obtained for routing an entire commodity). We present the state-of-the-art 4-approximation in this setting. One final extension is the maximum multicommodity demand flow problem where commodities have demands in addition to profits. We present the 48-approximation for this setting. The algorithm uses two parameterized rounding schemes to reduce a general demand problem to a series of unit demand problems, which admit a 4-approximation. The parameters are then optimized to achieve the approximation constant.

Acknowledgements

I would like to professor Bruce Shepherd for his expertise and enthusiasm during our many discussions as well as his continued support in my academic journey. His mentorship has been invaluable in my growth and the improvement of this thesis.

Table of Contents

Chapter 1

Introduction

We study the problem of *multicommodity flow* (or *multiflow* for short) on trees. In this problem, we are given a tree $G = (V, E)$ with integral edge capacities $\mathbf{c} \in 2^E$, and a commodity graph $H = (V, F)$ where F are the commodity pairs $s_i t_i$ we may select. We use P_i to denote the unique path of commodity i on G. A *flow* is then an assignment of non-negative flow values $f_i \in \mathbf{R}$ to each commodity path P_i, for $i \in [k]$. A *feasible* flow is a flow such that $\forall e, \sum_{i:e \in P_i} f_i \leq c_e$. The objective of the problem is to find a maximum feasible integral flow on G, i.e. each f_i is integral.

We also study two extensions to the maximum multiflow problem: multicommodity profit flows on trees and multicommodity demand flows on trees. In the *multicommodity profit flow problem*, each commodity is further associated with a *profit*, represented by the profit vector $\mathbf{w} \in \mathbf{Q}^F$. The *multicommodity demand flow problem* extends the multicommodity profit flow problem with *demands* for the commodities, represented by demand vector $\mathbf{d} \in \mathbf{Q}^F$. The goal in both settings is to find a feasible integral flow that maximizes the profit. We study both problems in the all-or-nothing profit regime where a profit w_i is obtained only by routing an entire demand d_i (entire unit for profit flows). For demand flows, we also make the *no-bottleneck assumption*, where $d_{max} \leq c_{min}$ for c_{min} the smallest edge-capacity and d_{max} the largest demand. This is a common assumption in the area [3].

It is worth noting that by limiting our scope to trees, we are equivalently studying the multiflow problems in the popular unsplittable flow regime where each commodity must be routed on a single path. Indeed each commodity can *only* be routed on one path on trees. The key algorithmic challenge is then in selecting which demands to route.

In Chapter 2, we begin by showing the equivalence between the maximum (cardinality) matching problem and the maximum multiflow problem on height-1 trees with unit edge-

capacities. This exhibits why our problem is of interest and hints at the difficulty of the maximum multicommodity flow problem even on trees. We then present a polynomial time algorithm for the unit capacity case inspired by this very equivalence of the height-1 setting to maximum matching. However, we next demonstrate that NP-hardness sets in quickly; the case of capacities in $\{1, 2\}$ is already APX-hard. From there, we show a constant 2-approximation for the problem on general capacities. These results appear in [5].

In Chapter 3, we study the maximum multicommodity profit flow problem. We present the state-of-the-art 4-approximation algorithm which uses an integral-vector decomposition technique [2]. This technique scales a optimal fractional flow (obtained by solving a linear program) into an integral flow to be decomposed into feasible integral flows that collectively attain the optimal profit overall. A bound on the number of these decomposed flows demonstrates that one flow must be a 4-approximation, by means of an averaging argument.

In Chapter 4, we present the state-of-the-art 46.168-approximation algorithm for the maximum multicommodity demand flow problem on trees [2]. To achieve this, a partition of the commodities by demand size into "large" and "small" demands is used. A demand d is "large" if $d \geq \beta d_{max}$ where β is a parameter chosen later. They give one rounding scheme for large demands, and a parameterized rounding scheme for small demands with a second parameter α. They show that by setting $\alpha = 0.442$ and $\beta = 0.311$, that one of the schemes is a 11.542Γ-approximation where Γ is the maximum integrality gap of any unit-demand problem instance. This can be combined with the 4-approximation algorithm in Chapter 3 since the multicommodity demand flow problem is equivalent to the multicommodity profit flow problem with unit demands. Overall, we have a 46.168-approximation for the maximum multicommodity demand flow problem.

Chapter 2

Max Multiflow

Given a capacitated tree $(G, \mathbf{u})$, and commodity graph $H = (V, F)$, we are interested in finding a maximum feasible integral flow on G. We denote this problem as MAXFLOWTREE. We begin with the restricted unit capacity setting where $\mathbf{u} = \mathbf{1}$. We denote these problems as MAXFLOWTREE1. We make the following assumption.

Assumption 1 (Leaf-to-leaf Demands)**.** *F consists only of edges between leaves of G.*

We make Assumption 1 without loss of generality since we can extend a leaf from the internal node that is incident only to that commodity in H with edge capacity 1. Note that this is not a polynomial time reduction.

We first show that the maximum multiflow problem on a star (a height-1 tree rooted at the center of the star) is equivalent to solving the maximum cardinality matching on H, the commodity graph.

The *maximum (cardinality) matching* problem seeks to find a maximum-sized matching on G. We say a node v is *saturated* by M if there exists some $e \in M$ incident on v. The maximum cardinality matching problem is well-studied and known to be efficiently solvable by Edmond's Blossom algorithm.

Theorem 2.1 ([4])**.** *The maximum cardinality matching problem on general graphs is solvable in polynomial time.*

Let MAXFLOWTREE1(T, H) with $V(T) = V(H)$ denote instance of MAXFLOWTREE1 on tree T and with commodity graph H. Also let $\phi(T, H)$ denote the optimal value of MAXFLOWTREE1(T, H). Finally, let $\nu(G)$ denote the size of the maximum cardinality

matching in G. The following result relates the maximum cardinality matching problem and the max multiflow problem on height-1 trees.

Proposition 2.2 ([5])**.** *For a height-1 tree $T = (V, E)$ rooted at r and commodity graph $H = (V, F)$, $\phi(T, H) = \nu(H)$.*

Proof. Let T, H be as per the proposition and f be a feasible flow to MAXFLOWTREE1(T, H). Consider the set of commodities on which f routes flow, denoted as $M \subseteq H$. Since the edge-capacities are unit and f is integral, at most one commodity on a non-root node $v \in V \backslash r$ can be routed by f. So M contains at most one edge incident to each node in H. Namely, M is a matching on H of size $|f|$.

Conversely, let M be a matching on H. Construct a multiflow f on T by letting $f_i = 1$ if its associated edge in H is in M. Then f is feasible since each leaf in T routes at most one unit of commodity. Each edge in M also adds exactly unit of flow to f. So f is a feasible flow of size $|M|$.

Since a feasible flow induces and matching and vice versa, we have $\nu(H) = \phi(T, H)$. □

Using this equivalence we can reduce MAXFLOWTREE1 on height-1 trees to a maximum cardinality matching problem, which admits a polynomial time algorithm by Theorem 2.1.

Corollary 2.3 ([5])**.** *There is a polynomial time algorithm which computes $\phi(T, H)$ for any height-1 tree $T = (V, E)$ rooted at r and commodity graph $H = (V, F)$.*

Figure 2.1: A MAXFLOWTREE1(T, H) instance. Solid lines are edges of T, dotted lines are edges of H (left). A maximum multiflow f on T and maximum matching on H in red (right).

2.1 General Trees with Unit Capacities

Problems on trees are generally more tractable than their counterparts on general graphs. The equivalence of max multiflow and maximum cardinality matching in Corollary 2.3 suggests that the problem is non-trivial even on trees. For general trees with unit capacities, the problem is equivalent to finding a maximum cardinality set of edge-disjoint paths between the commodities pairs. Using the equivalence in Corollary 2.3, Garg et al. develop a polynomial-time algorithm in this setting. The algorithm also uses an observation by Claude Berge on maximum matchings and *alternating paths* [1]. For a matching M, an (M-)alternating path is a path that alternates between M-edges and non M-edges. The follow is true about maximum matchings that *avoids* a node u.

Lemma 2.4 ([1])**.** *For a graph $G = (V, E)$, let $u \in V$ and M be a maximum cardinality matching on G that saturates u. Then there is a maximum cardinality matching M' that avoids u if and only if there exists an M-alternating path from u and to some unsaturated node. Furthermore, there is a polynomial time algorithm to find such a path if it exists.*

We can now present the algorithm by Garg et al.

Theorem 2.5. *There exists a polynomial time algorithm for* MAXFLOWTREE1*.*

Proof. Let T be a tree of height k rooted at some node r. Also let H be the commodity graph. We want to solve MAXFLOWTREE1(T, H). If $k = 1$ then we invoke **Proposition** 2.3 to find a maximum multiflow. So let $k > 1$.

Consider a subtree T_v rooted at a node v on the k-1st layer. Namely, T_v is a height-1 subtree of G. Note that T_v is connected to the rest of G by a single edge with unit capacity. Thus, only one node in T_v can choose to route its commodity (outside of T_v). So consider a commodity i with one endpoint $s \in T_v$ and the other $t \in T \backslash T_v$ outside of T_v (note both s and t are leaves in T by Assumption 1). We can only strictly improve the flow by routing st if there exists a maximum multiflow within T_v that doesn't route flow out of s. By **Proposition** 2.2 and **Proposition** 2.3, this is equivalent to the existence of a max matching on H that avoids s. Thus, we first compute a max matching on H, denoted as M_v. By **Proposition** 2.3, M_v induces a max multiflow within T_v. Using **Lemma** 2.4 and M_v, we also compute the set of all vertices S that can be avoided in a max matching on H and is incident to at least one commodity routing out of T_v. These commodities are precisely the ones that we may route outside of T_v to strictly improve the multiflow since there is a maximum multiflow within T_v that avoids them.

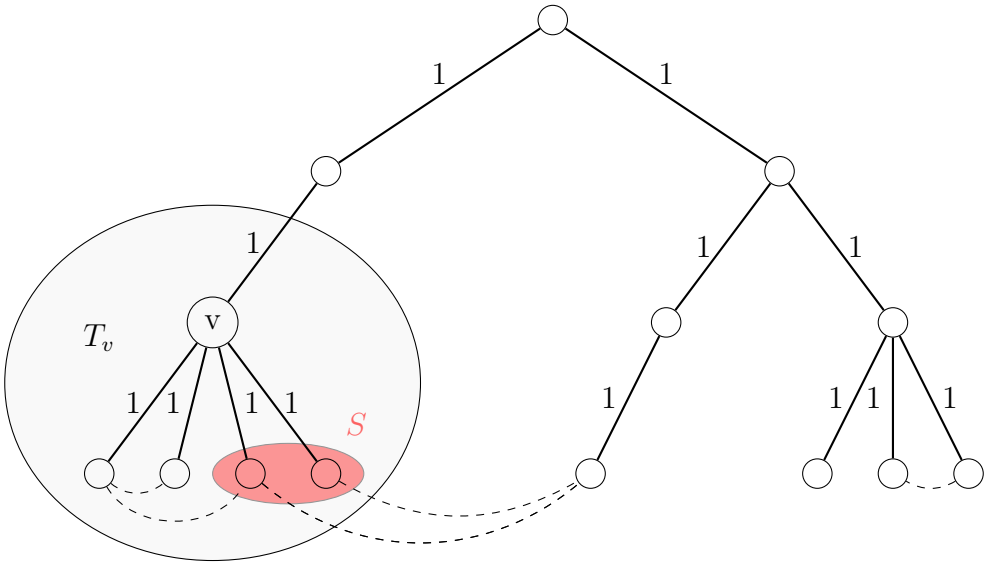

Figure 2.2: Example of T with $k = 3$. At most one of the two commodities going out of T_v can be routed.

We then perform a contraction of T_v into v, keeping only the commodities of nodes S routing out of T_v on v. Note that after contraction, Assumption 1 still holds. This procedure is done for every node, proceeding level by level from the lowest level, until a height-1 tree remains which can be solved by **Proposition** 2.3.

We now unwind the tree. For each contracted node $v \in T_v$, the max multiflow of the parent decides the commodity (if any) to route out of T_v, denoted as j. The commodities to route within T_v is then the ones that forms the max multiflow that avoids the endpoint of j in T_v.

In all, we perform $O(n)$ contractions, and polynomial work for each contraction to find a max matching and all vertices that can be avoided in a max matching. The unwinding step fixes the commodities to route, performing polynomial work for each vertex. Thus the algorithm is polynomial time. □

2.2 Hardness of Approximation

Despite an efficient algorithm for max integral multiflows with unit capacities, the problem quickly becomes difficult. In fact, it is APX-hard if we allow capacities in $\{1, 2\}$. We present

a reduction from the 3-dimensional matching problem.

Definition 2.6 (3-Dimensional Matching)**.** *Given* 3 *disjoint sets* X, Y, Z *of size* n *and a set of triples* $S = \{(x, y, z) : x \in X, y \in Y, z \in Z\}$*, a 3-dimensional matching (on* S*) is a subset* $S' \subseteq S$ *such that for* $(x, y, z), (x', y', z') \in S'$*, we have* $x \neq x', y \neq y'$*,and* $z \neq z'$*. We say that a set satisfying this property is disjoint. The maximum 3-dimensional matching problem seeks to find a largest* 3*-dimensional matching.*

This problem is known to be APX-hard even when each element occurs in at most d triples [8]. We denote this problem as MAXMATCH3$_{\leq d}$. The presentation relies on an *L-reduction* transformation, which we define next.

Definition 2.7 (L-reduction)**.** *For two classes of optimization problems* A *and* B *with cost functions* c_A *and* c_B*, an L-reduction from* A *to* B *is a polynomial time reduction* g *from instances of* A *to instances of* B *satisfying the following:*

1. $\exists \alpha \geq 1$ *such that for all instances* I *of* A, $\text{OPT}_B(g(I)) \leq \alpha \text{OPT}_A(I)$
2. $\exists \beta \geq 1$ *such that for every solution* y *of* $g(I)$*, we can find a solution* x *of instance* I *in polynomial time such that*

$$|\text{OPT}_A(I) - c_A(x)| \leq \beta |\text{OPT}_B(g(I)) - c_B(y)|$$

where $\text{OPT}_A(I)$ *and* $\text{OPT}_B(g(I))$ *are the optimal values of* I *and* $g(I)$ *respectively.*

Critically, if a problem A has a polynomial time constant-factor approximation and an L-reduction to problem B, then problem B has a PTAS as well. We defer the proof of result to Appendix A. We demonstrate an L-reduction from MAXMATCH3$_{\leq d}$ to MAXFLOWTREE with edge-capacities in $\{1, 2\}$. Then the APX-hardness of MAXFLOWTREE follows from the hardness of MAXMATCH3$_{\leq d}$ and Corollary A.2.

Theorem 2.8. MAXFLOWTREE *on trees with capacities in* $\{1, 2\}$ *is APX-hard.*

Proof. Let X, Y, Z be three disjoint sets such that $|X| = |Y| = |Z| = n$ and $S \subseteq X \times Y \times Z$. We show a reduction of the 3-dimensional matching problem on S to maximum integral multiflow with edge-capacities 1 or 2.

We construct a tree of height 3. The first layer contains a root node r. The second layer contains v_x for each element in $x \in X$, v_y for $y \in Y$ and v_z for $z \in Z$. Add a capacity-2 edge rv_x for $x \in X$. Add a capacity-1 edge rv_i for $i \in Y \cup Z$. Let p_x be the number of

triples of S containing x. We create a node $v_{x,i}$ for each occurence of x ($i \in [p_x]$) and add edges $v_x v_{x,i}$ with capacity 2. For each $v_{x,i}$, add two children $v_{x,i,L}$ and $v_{x,i,R}$ connected by capacity-1 edges.

Let $(x, y, z) \in S$ corresponds to be the ith occurence of x in S. Then add the commodities $(v_{x,i,L}, v_{x,i,R})$,$(v_{x,i,L}, v_y)$, and $(v_{x,i,R}, v_z)$. So we have $3|S|$ commodities in all.

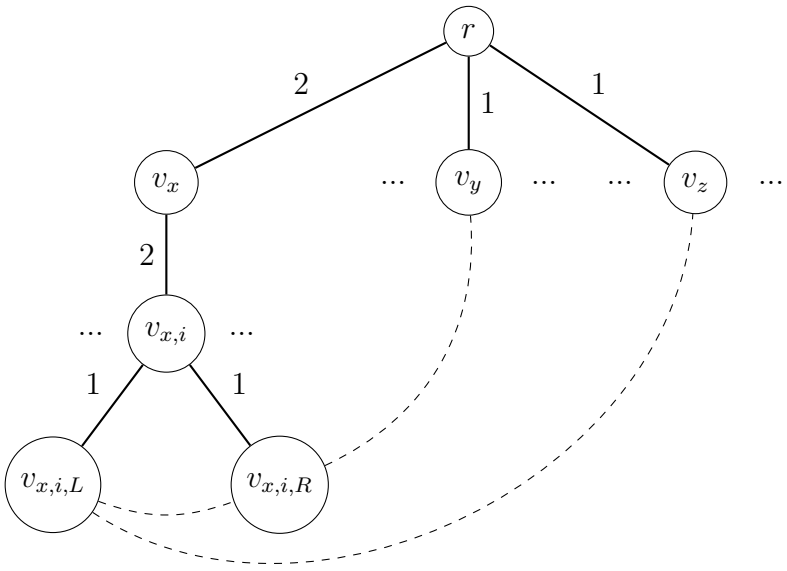

Figure 2.3: Contruction of MAXFLOWTREE instance from MAXMATCH3 instance. $(x, y, z) \in S$ is the ith occurrence of x in S.

Claim 2.9. *The* MAXFLOWTREE *instance has a flow of* $t+|S|$ *if and only if the* MAXMATCH3 *instance has* t *disjoint triples.*

Proof. Let $S' \subseteq S$ be a set of t disjoint triples and let T_x denote the subtree rooted at v_x. Then for every triple $(x, y, z) \in S'$ that is the ith occurence of x, route one unit of flow on $(v_{x,i,R}, v_y)$ and $(v_{x,i,L}, v_z)$. For $j \in [p_x]$ such that $i \neq j$, route one unit of flow on $(v_{x,j,L}, v_{x,j,R})$. For x that is not covered by S', route $(v_{x,i,L}, v_{x,i,R})$ for $i \in [p_x]$. It can be checked that this is a feasible flow by the disjointness of S'. Furthermore, if x is covered by S', then T_x routes $p_x + 1$ units of flow. It not, it routes p_x units of flow. In all, the

feasible flow has size

$$|f| = \sum_{x:S' \text{covers } x} (p_x + 1) + \sum_{x:S' \text{doesn't cover } x} p_x = t + \sum_{x \in X} p_x = t + |S|. \tag{2.1}$$

Now, consider an integral flow f of size $t+|S|$. For $x \in X$, observe that if f routes flow through two nodes $v_{x,i}$ and $v_{x,j}$ for $i \neq j$ (one unit each since the capacity of rv_x is 2), that the same flow value can be attained by routing one unit of $(v_{x,i,R}, v_{x,i,L})$ and $(v_{x,j,R}, v_{x,j,L})$ instead. Thus, the maximum flow attainable on commodities with at least one endpoint in T_x is $p_x + 1$ where a single $l \in [p_x]$ routes $(v_{x,l,L}, v_y)$ and $(v_{x,l,R}, v_z)$ for $(x, y, z) \in S$ and every other $k \in [p_x] - l$ routes $(v_{x,l,L}, v_y)$ and $(v_{x,l,R}, v_z)$. Since $\sum_{x \in X} p_x = |S|$ and $|f| = t + |S|$, there must be at least t elements $x \in X$ for which f routes $p_x + 1$ units of flow with one end point in T_x. This identifies t disjoint tuples in S'. □

Finally, it is left to check that the construction is an L-reduction. We let problem A be the maximum 3-dimensional matching problem where each element in X, Y, Z occurs in at most d triples and problem B be the maximum multiflow problem on trees with edge-capacities in $\{1, 2\}$. Also denote the reduction above as g. Then for a 3-dimensional matching instance I, $|S|/(3d-3) \leq \text{OPT}_A(I)$ because each triple intersects at most $3(d-1)$ other triples. Then

$$\text{OPT}_B(g(I)) = \text{OPT}_A(I) + |S| \leq (3d - 2)\text{OPT}_A(I), \tag{2.2}$$

so condition 1) is satisfied with $\alpha = 3d - 2$. Note that α is a constant because d is fixed in problem A. With a solution I of size t, we can also construct a solution $g(I)$ of size $t + |S|$. Hence,

$$|\text{OPT}_A(I) - t| = |\text{OPT}_B(g(I)) - |S| - t| = |\text{OPT}_B(g(I)) - (t + |S|)| \tag{2.3}$$

So condition 2) is satisfied with $\beta = 1$. □

2.3 2-approx for general capacities

We finish this section with a constant approximation algorithm for the general integral capacity case on trees. We study multicuts, which are the dual objects to the multicommodity flow polytope.

Definition 2.10 (Multicut for Trees)**.** *Given a tree $T = (V, E)$ and commodity set $H = (V, F)$, a multicut M is a collection of edges that disconnects the path in T between every $(v, u) \in H$. The capacity of a multicut, denoted by $c(M)$, is the sum of the capacity of the edges in M.*

Notice that the capacity of any multicut M is an upper bound on the value of any feasible integral flow f on the same graph T with commodities H. To see this, note

$$c(M) = \sum_{e \in M} c_e \geq \sum_{e \in M} (\sum_{i : e \in p_i} f(p_i)) = \sum_{i=1}^{k} \sum_{e \in p_i \cap M} f(p_i) \geq \sum_{i=1}^{k} f(p_i) = |f| \tag{2.4}$$

with the last inequality due to M separating the endpoints of every commodity. The 2-approximation we present is achieved by constructing a multicut M with capacity at most twice as large as some feasible integral multiflow of size f. This gives

$$\text{OPT}_{T,H} \leq c(M) \leq 2|f| \leq 2 \cdot \text{OPT}_{T,H} \tag{2.5}$$

where $\text{OPT}_{T,H}$ is the value of the maximum integral multiflow instance.

Theorem 2.11 ([5])**.** *There exists a 2-approximation algorithm for* MAXFLOWTREE*.*

Proof. The algorithm proceeds in two passes, with a bottom-up pass and a top-down pass. In the bottom-up pass, we greedily build a multiflow and a a multicut M to break this flow. The top-down pass then trims M to ensure it contains at most two edges on the path of any commodity. The 2-approximation then follows from

$$c(M) \leq \sum_{e \in M} c_e \leq \sum_{e \in M} (\sum_{i : e \in p_i} f(p_i)) = \sum_{i=1}^{k} \sum_{e \in p_i \cap M} f(p_i) \leq \sum_{i=1}^{k} 2f(p_i) = 2|f|. \tag{2.6}$$

Let r be the root of the tree. Throughout the proof, say an edge e_1 is an ancestor of edge e_2 if e_1 is on the path from r to e_2. An analogous definition also applies for nodes. We let $lca(u, v)$ denote the least common ancestor of nodes u and v.

Bottom-up pass. We iterate the nodes layer by layer from the bottom of the tree and greedily construct a flow f. For $v \in V$, we let T_v denote the subtree rooted at v. We say a commodity i is contained in $v \in V$ if it's path P_i in T is entirely in T_v.

When iterating to node v, if there is a commodity i contained in T_v such that P_i is not blocked by f, we push as flow as possible on P_i. Necessarily, at least one edge of P_i becomes saturated by f. Let the set of edges saturated on the iteration on v be $\text{FRONTIER}(v)$. Note that for $v \neq u$, $\text{FRONTIER}(v) \cap \text{FRONTIER}(u) = \emptyset$ since an edge can only be saturated once. Since the capacities are integral, f must also be integral.

Note that for every commodity i with endpoints s and t, if P_i is not already blocked before the iteration on $v = lca(s, t)$, then the procedure pushes flow to saturate some edge

on P_i on the iteration on v. So for every commodity i with endpoints s and t, there exists an edge $e \in P_i \cap \text{FRONTIER}(u)$ where $u = lca(s,t)$ or u is a descendent of $lca(s,t)$. Namely, $\bigcup_{v \in V} \text{FRONTIER}(v)$ is a multicut.

Top-bottom pass. We iterate layer by layer from the root down the tree to construct a multicut M'. At each $v \in V$, skip an edge $e \in \text{FRONTIER}(v)$ if there exists $e' \in M'$ such that e' is an ancestor of e. Otherwise add e to M'. To see that M' is a multicut, consider a commodity i and an edge $e \in \text{FRONTIER}(v)$ that blocks P_i. By construction, we only add e if there is no edge $e' \in M'$ between e and v already. In either case, there is an edge in M'

Note that M' is a multicut. To see this, consider a commodity i with endpoints s, t. From above, we know that there exists an edge $e \in P_i \cap \text{FRONTIER}(u)$ where $u = lca(s,t)$ or u is a descendent of $lca(s,t)$. When the procedure reaches u, we only exclude e only if there exists another edge $e' \in M'$ between e and u. In any case, there is an edge on P_i in M'. Next, we show two claims.

Claim 2.12. *Let i be a commodity with endpoints s, t that we send flow through. If $e \in$ FRONTIER(u) is on P_i, then either $u = lca(s,t)$ or u is an ancestor of $lca(s,t)$.*

Proof. Suppose $u \neq lca(s,t)$ and u is not an ancestor of v. Then u must have been considered before v in the bottom-up pass. Since $e \in \text{FRONTIER}(u)$, e is saturated when u is considered. Then flow cannot be sent through commodity i when it is considered, since e blocks P_i. This is a contradiction. □

Claim 2.13. *If u is an ancestor of v, then no edge of* FRONTIER(v) *is an ancestor of an edge* FRONTIER(u).

Proof. Let u be an ancestor of v. So v is considered first in the bottom-up pass. Thus, the flow f saturates any edge $e_v \in \text{FRONTIER}(v)$ before u is considered. So the iteration on u cannot saturate any edge e_u for which e_v is an ancestor, since any path involving e_u is blocked by e_v. □

Finally, consider any commodity i with endpoints s, t. We want to show that M' contains at most one edge between s and $lca(s,t)$. Suppose M' contains $e_1 \in \text{FRONTIER}(v_1)$ and $e_2 \in \text{FRONTIER}(v_2)$ between s and $lca(s,t)$ such that e_1 is the ancestor of e_2. Then by Claim 2.13, we know that v_1 is an ancestor of v_2. By Claim 2.12, we also know that v_1 and v_2 are ancestors of v (or is v). Thus e_1 occurs on the path from e_2 to v_2, so e_2 wouldn't have been included in M'. The same argument applies between t and $lca(s,t)$. Thus, the multicut M' has at most 2 edges for each routed flow, and the theorem follows from 2.6. □

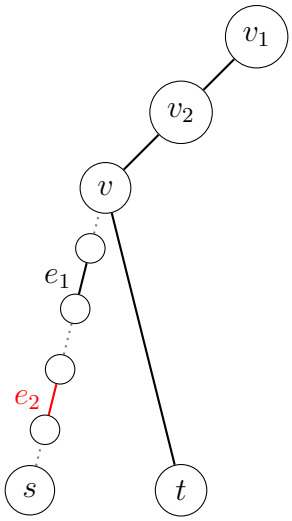

Figure 2.4: On iteration v_2 of top-down pass, e_2 is skipped

Chapter 3

Multicommodity Profit Flows

In the previous section, we presented a 2-approximation for the max integral multiflow on a tree with integer capacities. In this section, we consider a general version of the problem where profits are associated with each commodity. We study multicommodity profit flows in the all-or-nothing profit regime where exactly one unit of a commodity must be routed to obtain the associated profit. We again make Assumption 1 where commodities pairs are only between the leaves of the tree.

Similar to the maximum multicommodity flows problem, an instance of multicommodity profit flows consists of a capacitated tree $(G, \mathbf{u})$ and commodity graph $H = (V, F)$. We are additionally given a profit vector $\mathbf{w} \in \mathbf{Q}^F$ to represent the profit of the commodities. Then we may state the problem as the following integer linear program (ILP),

$$\max \sum_{f \in F} w_f x_f \qquad s.t. \tag{3.1}$$

$$\sum_{f : e \in P_f} x_f \leq u_e \qquad \forall e \in E \tag{3.2}$$

$$x_f \in \{0, 1\} \qquad \forall f \in F \tag{3.3}$$

where each variable x_i for $i \in [|F|]$ encodes the decision to route commodity i and P_f is the path of commodity f in G. It's worth noting that the multicommodity flow problem from Chapter 2 can also be formulated as an ILP by letting $x_f \in \mathbf{Z}_{\geq 0}$ and $w_f = 1$. A natural relaxation of the ILP is to allow $x_f \in [0, 1]$, $\forall f \in F$. This can be solved in polynomial time [6]. We refer to the ILP problem as IP and its relaxation as LP.

We demonstrate a 4-approximation algorithm for this problem setting [2]. Central to enabling profits is the following integer-vector decomposition theorem.

Theorem 3.1 (Decomposition of Multicommodity Flows [2]). *Let x a feasible solution to* LP*, and let $k \in \mathbf{N}$ be an integer such that kx is integral. Then there exists feasible solutions $z^1, ..., z^{4k}$ to* IP *such that*

$$kx \leq \sum_{i=1}^{4k} z^i. \tag{3.4}$$

Note that given these $z^1, ..., z^{4k}$, we have

$$kw^T x \leq \sum_{i=1}^{4k} \mathbf{w}^T z^i \tag{3.5}$$

Then by an averaging argument, we know $\exists z_{max} \in \{z^1, ..., z^{4k}\}$ such that

$$\frac{1}{4}\mathbf{w}^T x = \frac{1}{4k} k \mathbf{w}^T x \leq \frac{1}{4k} \sum_{i=1}^{4k} \mathbf{w}^T z^i \leq \mathbf{w}^T z_{max}. \tag{3.6}$$

So we obtain the following Corollary.

Corollary 3.2. *Given $z^1, ..., z^{4k}$ from Theorem 3.1, one of $z^1, ..., z^{4k}$ must be a $4k$-approximation of x.*

It is worth nothing that the overall approximation algorithm is parametrized on k and therefore potentially super-polynomial time to the input size. Chekuri et al. also present an analogous proof that is independent of k. We refer readers to the original paper [2] for details about the algorithm. The rest of this section is dedicated to proving Theorem 3.1.

3.1 Binned Tree Colouring

Theorem 3.1 is obtained by solving a generalized edge-colouring problem [2]. Specifically, an instance of the *binned tree colouring* problem which we define next.

Definition 3.3 (Edge-Colouring). *Given a graph G and $k \in \mathbf{N}$, an edge coluoring on G is a function $c : E \to [k]$. An edge-colouring c is proper if for any pair of incident edges $e, e' \in E$, we have $c(e) \neq c(e')$. Given a proper edge-colouring c on G, we say that 1) c colours G (or E), 2) $e \in E(G)$ has colour $c(e)$ (in c), and 3) G is k-edge-colourable. We also call $[k]$ the palette of c.*

Definition 3.4 (Colour Class)**.** *For a multigraph $G = (V, J)$, a map $c : J \to [k]$ for $k \in \mathbf{N}$, and a colour $j \in [k]$, the colour class of j is the multiset $C_j = \{e \in J : e \in c^{-1}(j)\} = \{s_1t_1, s_2t_2, ..., s_{|C_j|}t_{|C_j|}\}$ where $s_i, t_i \in V$. For a capacitated spanning tree T on V, the flow induced by C_j (in T) is obtained by pushing one unit of flow along the path of s_it_i in T, for $i \in [|C_j|]$. We say a C_j is routable (in T) if its induced flow is a feasible flow in T.*

Definition 3.5 (Fundamental Cut)**.** *For a graph $G = (V, E)$ and a spanning tree $T = (V, F)$, the fundamental cut of $e \in T$ is the set of edges of G in the cut formed by removing e in T.*

Definition 3.6 (Binned Tree Colouring)**.** *A binned tree colouring instance consists of*

1. *A capacitated tree (T, u) rooted at a node $v^* \in L(T)$, where $L(T)$ is the leaf set of T.*
2. *A multiset of undirected edges $J \subseteq L \times L$.*
3. *An integer $k \in \mathbf{N}$ such that $\forall e \in E(T)$, the number of edges of J in the fundamental cut of e is at most ku_e.*
4. *For each leaf node $v \neq v^*$, a partitioning of the edges in J incident to v into at most $u_{vp(v)}$ sets where $p(v)$ is the parent of v. Furthermore, each set in the partitioning has size in $[1, 2k)$. We denote this partitioning for v as $\mathbf{B}(v) = \{B_{v,1}, ..., B_{v,n_v}\}$. We also refer to a partition as a bin.*

We say a map $c : J \to [k]$ is a binned-colouring if each colour class is routable on T and if for any leaf node $v \neq v^*$, no two edges $e, e' \in B_{v,i}$ has $c(e) = c(e')$, for $i \in [n_v]$. If such a c exists, then we say the instance is k-bin-colourable.

Theorem 3.1 is obtained by a reduction from the multicommodity profit flow problem to the binned tree colouring problem. As we will see in Theorem 3.7, any binned tree colouring instance is $4k$-colourable. The flows induced by the $4k$ colour classes become the decomposed solutions to the original multicommodity profit flow instance. We proceed by showing the $4k$-colourability of the binned tree colouring problem.

Theorem 3.7 ([2])**.** *Every binned tree colouring instance is $4k$-colourable.*

Proof. Proceed by induction on the order of T in a binned-colouring instance (following notations in 3.6), denoted as $|T|$. In the base case, we have $|T| = 2$ and T is a single edge v^*v, for a non-root leaf v. Note that since v^* is both a root and leaf by definition, it is sufficient to consider the single edge as the base case. We know that $\mathbf{B}(v)$ is a partitioning

of size $n_v \leq u_{vv^*}$, where $|B_{v,i}| < 2k$ for $i \in [n_v]$. Thus, we can colour each bin using a palette of size $[2k]$. Each colour appears at most $n_v \leq u_{vv^*}$ times, so each colour class is routable. Hence the instance is $2k$-colourable and certainly $4k$-colourable.

Now, suppose for $n \in \mathbf{N}$, we have $|T| = n \geq 3$. We look for an internal node v_r whose children are all leaves, or a *remote node*. Note that any parent of the max-depth leaf node is necessarily a remote node since $|T| \geq 3$.

Let $L_{v_r} = \{v_1, v_2, ..., v_l\}$ be the children of v_r for some $l \in \mathbf{N}$. We construct a smaller binned-colouring instance to solve by induction. The new tree T' preserves edge capacities and contracts v_r and L_{v_r} into a single node v'_r. The new multigraph J' on the leaves of T' is obtained by the same contraction on J and deleting any self-loops on v'_r. We notice that the union of the partitionings on L_{v_r} forms a partitioning of the edges in J' on v'_r (after removing any contracted loops). We define a new partitioning $\mathbf{B}_{J'}(v'_r)$ by preserving any contracted bin of size at least k, and greedily packing other contracted bins until the bin has size at least k. Notice then that the bins at v'_r have size $[1, 2k)$, except maybe for the last bin in the greedy process. Let S be the number of J' edges crossing the fundamental cut of $v_r p(v_r)$. We note that we must have $n_{v'_r} \leq \frac{S}{k}$ since the packing scheme ensures each bin has size at least k if possible. Also recall $S \leq k u_{v_r p(v_r)}$ property 3 of Definition 3.6. So

$$kn_{v'_r} \leq S \leq k u_{v_r p(v_r)} = k u_{v'_r p(v'_r)} \implies n_{v'_r} \leq u_{v'_r p(v'_r)}. \tag{3.7}$$

For all other nodes $v \in L(T) \backslash \{v_r, v^*\}$, we keep $\mathbf{B}_{J'}(v) = \mathbf{B}_J(v)$. Indeed the contracted problem is a valid instance of binned tree colouring. By induction, we obtain a binned-colouring c on the new binned tree colouring instance with the palette [4k]. Each colour class of c is routable on T'. By construction, each original bin of $v_1, ..., v_l$ lies in exactly one new bin in the new partitioning. Thus, for $i \in \{1, ..., l\}$, there are at most $n_{v_i} \leq u_{v_i v_r}$ edges in J incident to v_i coloured C. So each colour class is routable in T.

Finally, we extend c to obtain a binned colouring of the original instance. For an uncoloured edge $e = v_i v_j$ for $v_i, v_j \in L_{v_r}$, let B_i and B_j be the bins that e lies in $\mathbf{B}_J(v_i)$ and $\mathbf{B}_J(v_j)$ respectively. For a bin B, also let $\textsc{coloured}(B)$ and $\textsc{uncoloured}(B)$ denote the edges in bin B on which c is defined and not defined, respectively. Then

$$|B_i| = |\textsc{coloured}(B_i)| + |\textsc{uncoloured}(B_i)| < 2k \tag{3.8}$$

$$\Rightarrow |B_i| = |\textsc{coloured}(B_i)| + |\textsc{uncoloured}(B_i) - \{e\}| + |\{e\}| < 2k \tag{3.9}$$

$$\Rightarrow |B_i| = |\textsc{coloured}(B_i)| + |\textsc{uncoloured}(B_i) - \{e\}| < 2k - 1. \tag{3.10}$$

The same is true of B_j. Thus, both B_i and B_j use fewer than $2k-1$ colours each. So there must exist 4 avaliable colours in $[4k]$ to colour e which we may pick arbitrarily for $c(e)$. We

also see that the unique path between v_i and v_j in T is (v_i, v_r, v_j). So the feasibility of colour class $c(e)$ in T follows from the feasibility of the colour class $c(e)$ in T' and by $n_{v_i} \leq u_{v_i v_r}$ and $n_{v_j} \leq u_{v_j v_r}$, since $c(e)$ is used at most once per bin. Overall c can be extended to a binned-colouring of the original binned tree colouring instance of size $[4k]$. □

3.2 Reduction From Multicommodity Profit Flows

We can use Theorem 3.7 to obtain Theorem 3.1 by reducing multicommodity profit flow to binned tree colouring.

Proof (of Theorem 3.7). Consider an instance of multicommodity profit flow with capacitated tree $(T, \mathbf{u})$ rooted at v^* with commodity graph H and profit vector $\mathbf{p}$. To construct the multiset of edges J, we take an optimal solution x to LP and $k \in \mathbf{N}$ such that kx is integral and add kx_f copies of commodity f to J. At each v, we construct a partitioning of the edges of J incident to v by greedily packing them so that each bin has between $[k, 2k)$ edges (except maybe the last). We also ensure that for each commodity f, that all kx_f copies of the commodity edge in J lies in the same bin. This is possible since there are at most $kx_f \leq k$ copies of each edge.

We also define the load on edge $e \in T$ as $l_e = \sum_{e \in P_f} x_f$. By the reduction, precisely kl_e edges lie in the fundamental cut of e in J. By feasibility of x, we know $kl_e \leq ku_e$. Thus, the number of edges of J in the fundamental cut of e is at most ku_e. Hence, J satisfies condition 3. of Defintion 3.6.

At each leaf v, the reduction yields $kl_{vp(v)}$ incident edges in J. By the partitioning scheme, the number of bins at v must satisfy $n_v \leq \frac{kl_{vp(v)}}{k}$, since we ensure each bin has size k if possible. So

$$kn_v \leq kl_{vp(v)} \leq ku_{vp(v)} \implies n_v \leq u_{vp(v)}. \tag{3.11}$$

Hence, the partitioning scheme satisfies condition 4. of Defintion 3.6. Overall the reduction produces a valid binned-colouring instance. By Theorem 3.7, we obtain $4k$ colour classes where each induces a feasible integral flow and routes at most one copy of each commodity. Since each edge in J belongs to some colour class, the sum of the induced flows by the colour classes is exactly the flow kx. Therefore, the induced flows obtain a profit of $\mathbf{w}^T kx$ and Theorem 3.1 holds.

Chapter 4

Demands

In this section we consider another extension to the maximum multiflow problem. In addition to profits, we associate each commodity with a demand. Formally, a problem instance additionally receives a demand-vector $\mathbf{d} \in Q^{|F|}$ where d_i represents how many units of commodity i we require. This problem is the *multicommodity demand flow* problem.

We similarly consider the all-or-nothing profit regime, where a profit is only obtained by routing a commodity's demand exactly. Similar to Chapter 2 and Chapter 3, we make Assumption 1 where demands only exist between the leaves of the tree. Furthermore, we make the *no-bottleneck assumption*.

Assumption 2 (No-Bottleneck Assumption)**.** *Any demand d_i is routable on any edge e: $d_i \leq u_e$.*

This is a common assumption in the area [3]. Note that multicommodity profit flows can be modelled with multicommodity demand flows by setting $\mathbf{d} = \mathbf{1}$. When all demands are identical, we say the demands are *uniform*. Otherwise, we say the demands are *general*. A small change to IP (3.1-3.3) gives an integer problem to capture the extension.

$$\max \sum_{f \in F} w_f x_f \qquad s.t. \tag{4.1}$$

$$\sum_{f : e \in P_f} d_f x_f \leq u_e \qquad \forall e \in E \tag{4.2}$$

$$x_f \in \{0, 1\} \qquad \forall f \in F. \tag{4.3}$$

We denote this integer problem as IP_d and its relaxation (allowing $x_f \in [0, 1]$) as LP_d.

The best known approximation algorithm for the multicommodity demand flow problem is a 46.168-approximation [2]. The rest of this chapter presents this algorithm.

4.1 Approach for General Demands

The approach of the 46.168-approximation algorithm relates the *integrality gap* of linear programs LP and LP_d.

Definition 4.1 (Integrality Gap)**.** *For an integer program* IP *and a relaxation* LP *with optimal values* OPT_I *and* OPT *respectively, the integrality gap between* IP *and* LP *is*

$$\frac{\text{OPT}}{\text{OPT}_I} \geq 1. \tag{4.4}$$

When it is clear, we may refer to the integrality gap between IP *and* LP *as the integrality gap of* IP *or the integrality gap of* LP*.*

Note that the integrality gap for any integer program and a relaxation to the program is greater than 1, since the feasible solutions to the integer program is a subset of the feasible solutions to the relaxation. To formalize the approach of the 46.168-approximation algorithm, we define two problem classes.

We let A be a $\{0,1\}$ matrix with m rows and n columns. For demands $d \in \mathbf{Q}_+^n$, we use the notation $A[d]$ to denote the matrix obtained by scaling the columns of A by d_i. We call a set $W \subseteq \mathbf{Q}^n$ *closed* if for any $w \in W$, the vector w' obtained by setting $w_i = 0$ for some $i \in [n]$ is also in W. We let $P(A, W)$ be the problems of form $\max\{wx : Ax \leq b, x \in [0,1]^n\}$ for some $w \in W$ and vector $b \in \mathbf{Z}_+^m$. We also let $P^{dem}(A, W)$ denote the class $\max\{wx : A[d]x \leq b, x \in [0,1]^n\}$ for some $w \in W$ and $d \in \mathbf{Q}_+^n$ with $d_{max} \leq b_{min}$ where d_{max} is the maximum entry of d and b_{min} the minimum of b. Note that $P(A, W)$ captures general any linear program with a $\{0,1\}$ constraint matrix.

We define the integrality gap of $P(A, W)$ as the supremum of the integrality gap of the problems in $P(A, W)$. We define the integrality gap of $P^{dem}(A, W)$ similarly. For a fixed A and W, the following theorem relates the integrality gap of $P(A, W)$ and $P^{dem}(A, W)$.

Theorem 4.2 ([2])**.** *Let A a $\{0,1\}$ matrix and W and closed set of integral vectors. If the integrality gap of $P(A, W)$ is Γ, then the integrality gap of $P^{dem}(A, W)$ is at most* 11.542Γ*.*

We use $\Pi(A, b, w) \in P(A, W)$ to denote the problem instance $\max\{wx : Ax \leq b, x \in [0,1]^n\}$. And $\Pi(A, b, w, d) \in P^{dem}(A, W)$ to be the problem instance $\max\{wx : A[d]x \leq$

$b, x \in [0,1]^n\}$. Given $S \subseteq [n]$, we denote A^S the matrix A restricted to the columns of S. Given a vector v, we denote by v^S the vector v restricted to the entries indexed by S.

In the proof of Theorem 4.2, we use two parameters α and $\beta \in (0,1)$. A demand f is *large* if $d_f \geq \beta d_{max}$, and is *small* otherwise. The proof demonstrates the construction of two solutions from small and large demands, respectively. It is then shown that one solution must obtain at least $\frac{1}{11.542\Gamma}$ of the profit. This technique of handling small and large demands with separate parameterized schemes is called *grouping-and-scaling* and can be attributed to [7].

We may use Theorem 4.2 to achieve an approximation for multicommodity demand flows as follows. Given an instance of multicommodity demand flow with tree $T = (V, E)$ and commodities $H = (V, F)$, we let $A = \{0,1\}^{F \times E}$ where $A_{ij} = 1$ if and only if e_j is on P_i. We also let $W = \mathbf{Q}^n$. Then $P(A, W)$ captures the multicommodity profit flow instances on T with commodities H with rational profit. Simiarly, $P^{dem}(A, W)$ captures the multicommodity demand flow instances with rational profits and rational demands on T and commodity graph H. By Corollary 3.2, we know that the integrality gap of $P(A, W)$ is at least 4. Namely, for our choice of A and W, we have $\Gamma \leq 4$. Thus, Theorem 4.2 gives that $P^{dem}(A, W)$ has integrality gap at most 46.168. Since Corollary 3.2 and Theorem 4.2 are shown constructively, we have a 46.168-approximation algorithm for the maximum multicommodity demand flow problem.

4.2 Parameterized Rounding Scheme for Large Demands

Lemma 4.3 (Parameterized Scheme for Large Demands)**.** *Let L be the set of large demands. Given a fractional solution x to $\Pi(A, b, w, d)$, there is an integral solution to $\Pi(A^L, b, w^L, d^L)$ of value at least $\frac{\beta}{2\Gamma} \sum_{f \in L} w_f x_f$.*

Proof. Given $x, L, \Pi(A^L, b, w^L, d^L)$, construct a new instance $\Pi(A^L, c, w^L, \hat{d})$ and feasible solution y^L where $\hat{d}$ is the vector of all d_{max} and $c := \lfloor \frac{b}{d_{max}} \rfloor d_{max}$ (the floor function here is applied to each vector-component). In particular, c_i is the smallest integer multiple of d_{max} that doesn't exceed b_i. Observe first that any solution to $\Pi(A^L, c, w^L, \hat{d})$ is a solution to $\Pi(A^L, b, w^L, d^L)$ with the same value. To see this, note that for a feasible solution z^L for $\Pi(A^L, c, w^L, \hat{d})$, we have

$$A^L[d^L]z^L \leq A^L[\hat{d}]z^L = d_{max} A^L z^L \leq c = \lfloor \frac{b}{d_{max}} \rfloor d_{max} \leq b. \tag{4.5}$$

Observe next that the constructed instance is a unit-demand instance since c is an integer multiple of $\lfloor \frac{b}{d_{max}} \rfloor$. Namely, $\Pi(A^L, c, w^L, \hat{d}) = \Pi(A^L, \lfloor \frac{b}{d_{max}} \rfloor, w^L) \in P(A, W)$.

Now, note for $y^L := \frac{\beta}{2} x^L$, we have

$$A^L[\hat{d}]y^L = \frac{d_{max}\beta}{2} A^L x^L \leq \lfloor \frac{b}{d_{max}} \rfloor \beta d_{max} = \beta c < c. \tag{4.6}$$

where the inequality is true by the feasibility of x and Assumption 2:

$$1 \leq \frac{b_i}{d_{max}} \implies \frac{b_i}{2d_{max}} \leq \lfloor \frac{b_i}{d_{max}} \rfloor \implies \frac{b_i}{2} \leq \lfloor \frac{b_i}{d_{max}} \rfloor d_{max}. \tag{4.7}$$

Thus, by our second observation, y^L is a feasible solution $\Pi(A^L, \lfloor \frac{b}{d_{max}} \rfloor, w^L)$. So there exists an integral solution y^L_* with object value $\frac{1}{\Gamma}(w^L)^T y^L = \frac{\beta}{2\Gamma} \sum_{f \in L} w_f x_f$. By our first observation, this solution is feasible for $\Pi(A^L, b, w^L, d^L)$ and we are done. □

4.3 Parameterized Rounding Scheme for Small Demands

Lemma 4.4 (Parameterized Scheme for Small Demands)**.** *Let S be the set of small demands. Given a fractional solution x to $\Pi(A, b, w, d)$, there is an integral solution to $\Pi(A^S, b, w^S, d^S)$ of value at least $\alpha(1 - \frac{\beta}{1-\alpha})\frac{1}{\Gamma} \sum_{f \in S} w_f x_f$.*

Proof. For $t \in \mathbf{N} \cup \{0\}$, let S_t be the demands d with $d \in (\alpha^{t+1}\beta d_{max}, \alpha^t \beta d_{max}]$. For $f \in S_t$, let $y^t_f = \alpha(1 - \frac{\beta}{1-\alpha})x_f$ and let $d^t_f = \alpha^t \beta d_{max}$. Given our fractional solution x, We define the load on constraint i as $l^t_i := \sum_{f \in S_t} A_{if} d_f x_f$. Also let c^t_i be the smallest integer multiple of $\alpha^t \beta d_{max}$ larger than $(1 - \frac{\beta}{1-\alpha})l^t_i$. Note that

$$c^t_i \leq \alpha^t \beta d_{max} + (1 - \frac{\beta}{1-\alpha}) l^t_i. \tag{4.8}$$

Then we have constructed problem a unit demand problem $\Pi(A^{S_t}, c^t, d^t, w^{S_t}) = \Pi(A^{S_t}, \frac{c^t}{\alpha^t \beta d_{max}}, w^{S_t})$

for each t. Note that y_f^t is feasible for this problem.

$$\sum_{f\in S_t} A_{if} d_f^t y_f^t \tag{4.9}$$

$$= \sum_{f\in S_t} A_{if}\big(\alpha^t \beta d_{max}\big)\big(\alpha(1-\frac{\beta}{1-\alpha})x_f\big) \qquad \to \text{(definition of } d^t, y^t) \tag{4.10}$$

$$=(1-\frac{\beta}{1-\alpha})\sum_{f\in S_t} A_{if}\big(\alpha^{t+1}\beta d_{max}\big)x_f \tag{4.11}$$

$$\leq (1-\frac{\beta}{1-\alpha})\sum_{f\in S_t} A_{if} d_f x_f \qquad \to \text{(definition of } d^t) \tag{4.12}$$

$$=(1-\frac{\beta}{1-\alpha})l_i^t \qquad \to \text{(definition of } l^t) \tag{4.13}$$

$$\leq c_i^t \qquad \to \text{(definition of } c^t). \tag{4.14}$$

Thus, there exists integral solution z^t for each t such that

$$\frac{1}{\Gamma}\sum_{f\in S_t} w_f y_f^t = \frac{1}{\Gamma}\alpha(1-\frac{\beta}{1-\alpha})\sum_{f\in S_t} w_f x_f \leq \sum_{f\in S_t} w_f z_f^t. \tag{4.15}$$

Note the following is true about z^t.

$$\sum_{f\in S_t} A_{if} d_f^t z_f^t \leq c_i^t \leq \alpha^t \beta d_{max} + (1-\frac{\beta}{1-\alpha})\big(\sum_{f\in S_t} A_{if} d_f x_f\big) \tag{4.16}$$

where the first inequality is due to the feasibility of z^t and the second inequality is due to Equation (4.8) and the definition of l^t. Summing over t gives

$$\sum_{f\in S} A_{if} d_f z_f \leq c_i \leq \frac{\beta}{1-\alpha} d_{max} + (1-\frac{\beta}{1-\alpha})\big(\sum_{f\in S} A_{if} d_f x_f\big) \tag{4.17}$$

By Assumption 2 and the feasibility of x, we have

$$\frac{\beta}{1-\alpha} d_{max} + (1-\frac{\beta}{1-\alpha})\big(\sum_{f\in S} A_{if} d_f x_f\big) \leq \frac{\beta}{1-\alpha} b_i + (1-\frac{\beta}{1-\alpha}) b_i = b_i. \tag{4.18}$$

Thus, the sum of all $z := \sum_{t\in \mathbf{N}\cup\{0\}} z^t$ is a feasible solution to our original problem. By Equation (4.15), we know z has profit has least

$$\frac{1}{\Gamma}\alpha(1-\frac{\beta}{1-\alpha})\sum_{f\in S} w_f x_f \leq \sum_{f\in S} w_f z_f \tag{4.19}$$

as required. □

Lemma 4.5 (Optimizing α)**.** *For fixed β, the optimal α is chosen so that $\alpha = 1 - \sqrt{\beta}$.*

Proof. For fixed β, we are guaranteed a small demand solution of size

$$f(\alpha) := \frac{1}{\Gamma}\alpha(1 - \frac{\beta}{1-\alpha})\sum_{f\in S} w_f x_f. \tag{4.20}$$

This is a one dimensional optimization problem. Thus, using first-order calculus, it can be checked that Equation 4.20 is maximized when $\alpha = 1 - \sqrt{\beta}$. □

Corollary 4.6 (Large and Small Demand Solutions)**.** *For x an optimal solution to LP_d, $\frac{1}{\Gamma}$ the integrality gap of $P(A, W)$, and S and L the set of small and large-demand commodities (namely, $S \cup L = F$), there exists two solutions to IP_d with objective values $\frac{\beta}{2\Gamma}\sum_{f\in L} w_f x_f$ and $(1 - 2\sqrt{\beta} + \beta)\frac{1}{\Gamma}\sum_{f\in S} w_f x_f$, respectively.*

4.4 Constant Approximation for General Demands

Finally, we show Theorem 4.2 by combining Lemma 4.3 and Lemma 4.4.

Proof of Theorem 4.2. Let $\epsilon \in [0, 1]$ be the proportion of profits obtained by small demands. Namely, $\sum_{f\in S} w_f x_f = \epsilon\ \mathbf{w}^T x$ and $\sum_{f\in L} w_f x_f = (1-\epsilon)\ \mathbf{w}^T x$. Then by Corollary 4.6, we have two solutions of sizes $(1-\epsilon)\frac{\beta}{2\Gamma}\ \mathbf{w}^T x$ and $\epsilon(1 - 2\sqrt{\beta} + \beta)\frac{1}{\Gamma}\ \mathbf{w}^T x$, respectively. Since we have no control over the distribution of profits, ϵ, we want to optimize

$$\max_{\beta\in(0,1)} \min_{\epsilon\in[0,1]} (\max\{(1-\epsilon)\frac{\beta}{2}, \epsilon(1 - 2\sqrt{\beta} + \beta)\}) \tag{4.21}$$

Let

$$P(\beta) := \min_{\epsilon\in[0,1]} (\max\{(1-\epsilon)\frac{\beta}{2}, \epsilon(1 - 2\sqrt{\beta} + \beta)\}) \tag{4.22}$$

denote the inner minimization problem.

Proposition 4.7. *For a fixed β, $P(\beta)$ is optimal when $\epsilon \in [0, 1]$ is chosen so that*

$$(1-\epsilon)\frac{\beta}{2} = \epsilon(1 - 2\sqrt{\beta} + \beta)$$

Proof. Suppose for $P(\beta)$ is optimized by some ϵ such that $(1-\epsilon)\frac{\beta}{2} < \epsilon(1-2\sqrt{\beta}+\beta)$. Then $\max\{(1-\epsilon)\frac{\beta}{2}, \epsilon(1-2\sqrt{\beta}+\beta)\} = \epsilon(1-2\sqrt{\beta}+\beta) > ((1-\epsilon)\frac{\beta}{2})$. Let $D = (\epsilon(1-2\sqrt{\beta}+\beta)) - ((1-\epsilon)\frac{\beta}{2}) > 0$ be this difference. Let $\delta \in \mathbf{R}_+$ be some value such that

$$0 < \delta < \min(\frac{D}{2(1-2\sqrt{\beta}+\beta)}, \frac{D}{\beta}) \tag{4.23}$$

and consider $\epsilon^* = \epsilon - \delta$. Then

$$\begin{aligned}
(1-\epsilon)\frac{\beta}{2} &< (1-\epsilon^*)\frac{\beta}{2} && (\text{by } \delta > 0) && (4.24)\\
&= (1-\epsilon)\frac{\beta}{2} + \delta\frac{\beta}{2} && && (4.25)\\
&< (1-\epsilon)\frac{\beta}{2} + \frac{D}{2} && (\text{by } \delta < \frac{D}{\beta}) && (4.26)\\
&\leq \epsilon(1-2\sqrt{\beta}+\beta) - \frac{D}{2} && (\text{Definition of } D) && (4.27)\\
&< \epsilon(1-2\sqrt{\beta}+\beta) - \delta(1-2\sqrt{\beta}+\beta) && (\text{by } \delta < \frac{D}{2(1-2\sqrt{\beta}+\beta)}) && (4.28)\\
&= \epsilon^*(1-2\sqrt{\beta}+\beta) && && (4.29)\\
&< \epsilon(1-2\sqrt{\beta}+\beta) && (\text{by } \delta > 0). && (4.30)
\end{aligned}$$

So

$$\max\{(1-\epsilon^*)\frac{\beta}{2}, \epsilon^*(1-2\sqrt{\beta}+\beta)\} < \epsilon(1-2\sqrt{\beta}+\beta) = \max\{(1-\epsilon)\frac{\beta}{2}, \epsilon(1-2\sqrt{\beta}+\beta)\}$$

with the first inequality due to Equation (4.24) and (4.30), a contradiction. It can be checked that for $(1-\epsilon)\frac{\beta}{2} > \epsilon(1-2\sqrt{\beta}+\beta)$ the same argument holds by setting $D = ((1-\epsilon)\frac{\beta}{2}) - (\epsilon(1-2\sqrt{\beta}+\beta)) > 0$ and $\epsilon^* = \epsilon + \delta$. □

Corollary 4.8. *Problem (4.21) is equivalent to*

$$\max_{\beta\in(0,1)} \{(1 - \frac{\beta}{2-4\sqrt{\beta}+3\beta})\frac{\beta}{2}\} \tag{4.31}$$

Proof. For any fixed β, Lemma 4.7 tells us that $P(\beta)$ is optimized when $(1-\epsilon)\frac{\beta}{2} =$

$\epsilon(1 - 2\sqrt{\beta} + \beta)$. Solving for ϵ gives us that:

$$\epsilon(1 - 2\sqrt{\beta} + \beta) = (1 - \epsilon)\frac{\beta}{2} \tag{4.32}$$

$$\epsilon - 2\epsilon\sqrt{\beta} + \beta\epsilon = \frac{\beta}{2} - \frac{\beta\epsilon}{2} \tag{4.33}$$

$$\epsilon - 2\epsilon\sqrt{\beta} + \beta\epsilon + \frac{\beta\epsilon}{2} = \frac{\beta}{2} \tag{4.34}$$

$$\epsilon(1 - 2\sqrt{\beta} + \beta + \frac{\beta}{2}) = \frac{\beta}{2} \tag{4.35}$$

$$\epsilon = \frac{\beta}{2 - 4\sqrt{\beta} + 3\beta} \tag{4.36}$$

Substituting this expression into Problem (4.21) yields result. □

Using a numerical solver, Problem (4.21) can be resolved for $\beta \in (0, 1)$. The optimal solution at $\epsilon = 0.442494$, $\beta = 0.310814$ generates an objective of $\frac{0.08664}{\Gamma} = \frac{1}{11.542\Gamma}$.

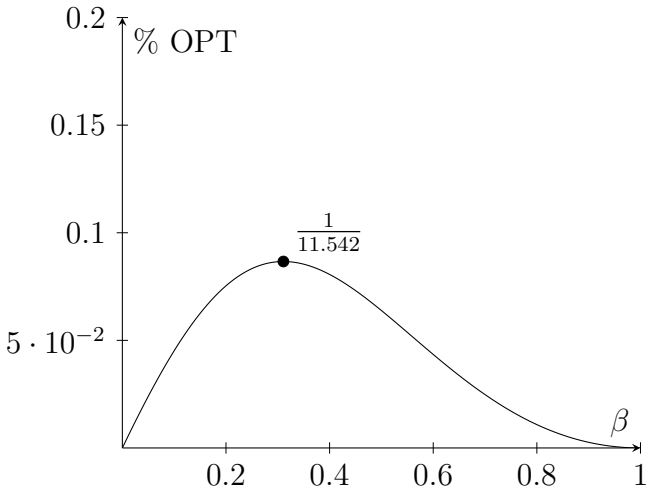

Figure 4.1: Optimal Choice of β for Problem 4.21

References

[1] Claude Berge. Two theorems in graph theory. *Proceedings of the National Academy of Sciences*, 43(9):842–844, 1957.

[2] Chandra Chekuri, Marcelo Mydlarz, and F. Bruce Shepherd. Multicommodity demand flow in a tree and packing integer programs. *ACM Trans. Algorithms*, 3(3):27–es, August 2007.

[3] Yefim Dinitz, Naveen Garg, and Michel X. Goemans. On the single-source unsplittable flow problem. *Combinatorica*, 19(1):17–41, 1999.

[4] Jack Edmonds. Paths, trees, and flowers. *Canadian Journal of Mathematics*, 17:449–467, 1965.

[5] Naveen Garg, Vijay V. Vazirani, and Mihalis Yannakakis. Primal-dual approximation algorithms for integral flow and multicut in trees. *Algorithmica*, 18(1):3–20, 1997.

[6] Martin Grötschel, László Lovász, and Alexander Schrijver. *The Ellipsoid Method*, pages 64–101. Springer Berlin Heidelberg, Berlin, Heidelberg, 1993.

[7] Stavros G. Kolliopoulos and Clifford Stein. Approximation algorithms for single-source unsplittable flow. *SIAM Journal on Computing*, 31(3):919–946, 2001.

[8] Christos H. Papadimitriou and Mihalis Yannakakis. Optimization, approximation, and complexity classes. *Journal of Computer and System Sciences*, 43(3):425–440, 1991.

APPENDICES

Appendix A

L-reductions and APX-hardness

A.1 Constant Approximations

Many decision problems (problems where the output is 'yes' or 'no') in computer science are hard. One well-known class of decision problems is NP. Practically, NP contains the decision problems that admit a certificate that can be verified in polynomial time if the answer is 'yes'. An example is the 3-dimensional matching problem (see Definition 2.6) where for sets X, Y, Z with $|X| = |Y| = |Z| = n$ and $S \subseteq X \times Y \times Z$, we ask: does there exist a 3-dimensional matching of S of size $k \in \mathbf{N}$ ($k \leq n$)? A certificate for this problem would be a matching of size at least k, for which we can feasibility in polynomial time to the input size.

Many NP problems have analogous optimization versions. For the 3-dimensional matching problem, the analogous optimization problem is the *maximum* 3-dimensional matching problem where we look for the maximum sized 3-dimensional matching of S instead. This set of problems is classified as NPO, which are optimization problems that look for the optimal solution among a set of feasible solutions with an underlying NP problem. Unlike algorithms for NP problems, it may be beneficial for optimization algorithms to output a 'good' intermediate solution even if it cannot find an optimization solution. For a maximization problem with constant function c and optimal value OPT, a polynomial time constant-factor approximation with constant $r \geq 1$ is an algorithm that outputs a feasible solution x with value $c(x) \geq \frac{1}{r}\text{OPT}$ in polynomial time (to input size). The class of optimization problems that admit such as algorithm is APX. It is known that a problem A is APX-hard if there exists of a polynomial time constant-factor approximation for A if and only if P=NP [8].

A.2 APX-hardness and L-reduction

An L-reduction (see Definition 2.7) is a transformation that preserves the distance between feasible and optimal solutions. We formalize this in the following lemma.

Lemma A.1 ([8]). *For maximization problems A and B with cost functions c_A and c_B, if B has a polynomial time constant-factor approximation and there is an L-reduction from A to B, then A also has a polynomial time constant-factor approximation.*

Proof. Let ALG_B be a PTAS for B with approximation constant $r \geq 1$. We denote our L-reduction as g. Let I be an instance of problem A and $g(I)$ the instance of problem B after applying g. We use ALG_B to find a solution y to $g(I)$ with approximation ratio

$$\frac{\text{OPT}_B(g(I))}{c_B(y)} = \frac{1}{r}.$$

By definition of 2.7, we also find a solution x to I in polynomial time such that

$$\text{OPT}_A(I) - c_A(x) \leq \beta(\text{OPT}_B(g(I)) - c_B(y)).$$

Note we safety remove the absolute value signs since we have a maximization problem. Then

$$\frac{c_A(x)}{\text{OPT}_A(I)} \geq \frac{\text{OPT}_A(I) - \beta(\text{OPT}_B(g(I)) - c_B(y))}{\text{OPT}_A(I)} \geq 1 - \alpha\beta\left(\frac{\text{OPT}_B(g(I)) - c_B(y)}{\text{OPT}_B(g(I))}\right)$$

$$= 1 - \alpha\beta(1 - r) = \alpha\beta r.$$

So A has a polynomial time constant-factor approximation, with factor $\alpha\beta r$. □

Corollary A.2 (Converse to A.1). *For maximization problems A and B, if*

1. *A does not have a polynomial time constant-factor approximation and*
2. *there exists an L-reduction from A to B,*

then B does not have a polynomial time constant-factor approximation unless P=NP.

Thus, by our reduction from $\text{MAXMATCH3}_{\geq d}$ to MAXFLOWTREE, we demonstrate Theorem 2.8.